

Deep Learning for Agents

Intelligent Systems II

Mário Antunes

Universidade de Aveiro

2026

The Deep Learning Revolution

The Connectionist Dream

- **Connectionism:** A movement in cognitive science that models mental phenomena as emergent processes of interconnected networks of simple units.
 1. **1943:** McCulloch and Pitts model of a neuron.
 2. **1958:** Rosenblatt's Perceptron.
 3. **1986:** Rumelhart, Hinton, and Williams popularize Backpropagation.
 4. **2012:** AlexNet wins ImageNet, starting the "Deep Learning" era.

Deep Learning vs. Classical ML

- **Generalization:** DL often scales better with dataset size.
- **Representation Learning:** Learning the optimal way to represent the input for a task.
- For example in **Classical ML** image processing we used:
 - **Hand-crafted Features:** SIFT, HOG, manual statistics. (Classical)
 - **Learned Features:** Filters that evolve to detect edges, textures, then objects. (Deep)

The Hierarchy of Features

- **Low-level:** Edges, gradients, color blobs.
- **Mid-level:** Patterns, corners, basic shapes.
- **High-level:** Object parts (eyes, wheels), entire objects.
- **Final Layer:** Abstract concepts (Cat, Car, Winning Move).

- **Compute:** DL capability is highly correlated with FLOPS.
- **Data:** Large models require huge datasets to avoid overfitting.
- **Parameters:** Millions or Billions of weights allow for high-capacity learning.
- **The Sweet Spot:** Balancing model size, data volume, and training time.

Limitations of Deep Learning

- **Black Box:** Hard to interpret why a decision was made.
- **Data Hunger:** Needs thousands/millions of labeled examples.
- **Compute Cost:** Training large models requires expensive hardware.
- **Adversarial Vulnerability:** Small, invisible perturbations can confuse the model.

The Multi-Layer Perceptron (MLP)

Anatomy of a Neuron

- Input vector: $\mathbf{x} = [x_1, x_2, \dots, x_n]$
- Weights: $\mathbf{w} = [w_1, w_2, \dots, w_n]$
- Bias: b (allows the activation to shift).
- Linear Sum: $z = \mathbf{w}^T \mathbf{x} + b$
- Activation Function: $a = \sigma(z)$ (Adds non-linearity).

Why Non-Linearity?

- A composition of linear functions is still linear.
- Without non-linear activations, a 100-layer network is mathematically equivalent to a 1-layer network.
- Non-linearity allows the network to approximate ANY continuous function (Universal Approximation Theorem).

Activation: Sigmoid and Tanh

- **Sigmoid:** $\sigma(z) = \frac{1}{1+e^{-z}}$. Used for probabilities.
 - *Issue:* Vanishing gradients for very high or low z .
- **Tanh:** $\tanh(z) = 2\sigma(2z) - 1$. Centered at 0.
 - *Benefit:* Often converges faster than sigmoid.

Activation: ReLU and Variants

- **ReLU:** $\max(0, z)$. Fast to compute, sparse activation.
- **Leaky ReLU:** $\max(0.01z, z)$. Prevents “dead neurons”.
- **ELU:** Smoother transition, can be more robust.
- **Swish:** $z \cdot \sigma(z)$. Modern, used in EfficientNet.

Initializing Weights

- **Zero Initialization:** Fails (all neurons in a layer learn the same thing).
- **Random Initialization:** Breaks symmetry.
- **Xavier/Glorot:** Keeps variance consistent across layers (good for Sigmoid/Tanh).
- **He Initialization:** Optimized for ReLU.

Backpropagation Math

Forward Pass Step-by-Step

For a layer l :

1. Input: $\mathbf{a}^{(l-1)}$
2. Weight Matrix: $\mathbf{W}^{(l)}$
3. Bias Vector: $\mathbf{b}^{(l)}$
4. $z_j^{(l)} = \sum_k w_{jk}^{(l)} a_k^{(l-1)} + b_j^{(l)}$
5. $a_j^{(l)} = \sigma(z_j^{(l)})$

The Objective: Gradient of Loss

We want to find:

$$\nabla_{\mathbf{w}, \mathbf{b}} J = \left[\frac{\partial J}{\partial w_{jk}^{(l)}}, \frac{\partial J}{\partial b_j^{(l)}} \right]$$

Using the chain rule:

$$\frac{\partial J}{\partial w_{jk}^{(l)}} = \frac{\partial J}{\partial a_j^{(l)}} \frac{\partial a_j^{(l)}}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial w_{jk}^{(l)}}$$

Defining the Error δ

Let $\delta_j^{(l)} = \frac{\partial J}{\partial z_j^{(l)}}$.

Since $a_j^{(l)} = \sigma(z_j^{(l)})$, then $\frac{\partial a_j^{(l)}}{\partial z_j^{(l)}} = \sigma'(z_j^{(l)})$.

Then:

$$\delta_j^{(l)} = \frac{\partial J}{\partial a_j^{(l)}} \sigma'(z_j^{(l)})$$

The Weight Gradient

Since $z_j^{(l)} = \dots + w_{jk}^{(l)} a_k^{(l-1)} + \dots$, then $\frac{\partial z_j^{(l)}}{\partial w_{jk}^{(l)}} = a_k^{(l-1)}$.

Substituting back:

$$\frac{\partial J}{\partial w_{jk}^{(l)}} = \delta_j^{(l)} a_k^{(l-1)}$$

The gradient is simply the error at the current node times the activation of the previous node.

Backpropagating to Previous Layers

How to find $\delta_j^{(l)}$ from $\delta^{(l+1)}$?

$$\delta_j^{(l)} = \frac{\partial J}{\partial z_j^{(l)}} = \sum_k \frac{\partial J}{\partial z_k^{(l+1)}} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}}$$

$$\delta_j^{(l)} = \sum_k \delta_k^{(l+1)} \frac{\partial z_k^{(l+1)}}{\partial a_j^{(l)}} \frac{\partial a_j^{(l)}}{\partial z_j^{(l)}}$$

Since $z_k^{(l+1)} = \sum_m w_{km}^{(l+1)} a_m^{(l)} + b_k^{(l+1)}$, then

$$\frac{\partial z_k^{(l+1)}}{\partial a_j^{(l)}} = w_{kj}^{(l+1)}.$$

Final Backprop Formulas

1. Output error: ${}^{(L)} = \nabla_{\mathbf{a}^{(L)}} J \odot \sigma'(\mathbf{z}^{(L)})$
2. Hidden error: ${}^{(l)} = ((\mathbf{W}^{(l+1)})^T {}^{(l+1)}) \odot \sigma'(\mathbf{z}^{(l)})$
3. Weight grad: $\frac{\partial J}{\partial \mathbf{W}^{(l)}} = {}^{(l)} (\mathbf{a}^{(l-1)})^T$
4. Bias grad: $\frac{\partial J}{\partial \mathbf{b}^{(l)}} = {}^{(l)}$

The Vanishing Gradient Problem

- For deep networks, gradients are multiplied many times.
- If $\sigma'(z) < 1$ (like Sigmoid), the gradient shrinks exponentially.
- Result: Early layers don't learn.
- Solution: ReLU, Batch Normalization, Residual Connections (ResNet).

Advanced Optimizers

Stochastic Gradient Descent (SGD)

- Update using one sample (or a mini-batch) at a time.
- $w \leftarrow w - \eta \nabla J_i$
- Fast, but noisy.
- Noise can help “jump out” of local minima.

- Keep a running average of gradients.
- $v_t = \gamma v_{t-1} + \eta \nabla J$
- $w \leftarrow w - v_t$
- “Physics” analogy: a ball rolling down a hill gathers momentum.

- Scale learning rate by a moving average of squared gradients.
- $E[g^2]_t = 0.9E[g^2]_{t-1} + 0.1g_t^2$
- $w \leftarrow w - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$
- Dampens oscillations in high-gradient directions.

- Adaptive Moment Estimation.
- Tracks first moment (m_t , momentum) and second moment (v_t , RMSProp).
- Includes bias correction for early steps.
- Very robust to hyperparameter choices.

Convolutional Networks (CNN)

The Problem with Images in MLP

- A 1000×1000 image has 1M pixels.
- One hidden layer with 1000 units = 1 Billion parameters.
- MLPs don't care about spatial proximity.
- Solution: Convolutions.

Convolution Operation

- A filter (kernel) of size $k \times k$ slides over the input.
- Element-wise multiplication and sum.
- Feature Map: The resulting grid.
- Stride: How many pixels the filter moves.
- Padding: Adding zeros to the border to keep size.

CNN Architecture (Tikz)



The Convolution Operation

- Given an image I and a kernel K of size $m \times n$:

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n)$$

- Feature Extraction:** Filters learn to detect specific patterns (edges, textures).
- Sparse Connectivity:** Each neuron in a conv layer is only connected to a small region of the input.
- Parameter Sharing:** The same kernel weights are used across the entire image.

Pooling and Translation Invariance

- **Max Pooling:** Selects the maximum value in a window.

$$a_{i,j}^{(l)} = \max_{m,n \in \text{window}} a_{i+m,j+n}^{(l-1)}$$

- **Purpose:**
 - Reduces spatial dimensions (downsampling).
 - Provides translation invariance (a small shift in input doesn't change output).
 - Reduces computational cost and prevents overfitting.

- **Backprop through Max Pooling:**
 - Gradient only flows to the neuron that was the maximum during the forward pass.
 - All other gradients in the window are zero.
- **Backprop through Convolution:**
 - Equivalent to a convolution with a rotated kernel.
 - Weight gradient: $\frac{\partial J}{\partial K} = \text{Input} * \delta^{(l)}$
 - Input gradient: $\delta^{(l-1)} = \delta^{(l)} * \text{Rotated}(K)$

Sequence Models

- RNNs process sequences $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$.
- **State Update Equation:**

$$\mathbf{h}_t = \sigma(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$$

- **Output Equation:**

$$\mathbf{y}_t = \text{softmax}(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y)$$

- $\mathbf{h}_t$ acts as a “memory” of previous time steps.

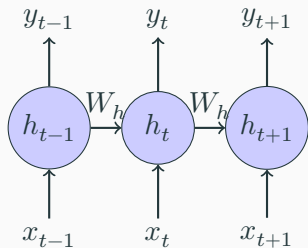
Backpropagation Through Time (BPTT)

- The gradient at time t depends on all previous time steps $1, \dots, t - 1$.
- Using the chain rule over time:

$$\frac{\partial J_t}{\partial \mathbf{W}_h} = \sum_{k=1}^t \frac{\partial J_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial \mathbf{h}_k}{\partial \mathbf{W}_h}$$

- **Vanishing Gradient:** If eigenvalues of $\mathbf{W}_h$ are < 1 , the product of derivatives goes to zero.
- **Exploding Gradient:** If eigenvalues are > 1 , the product goes to infinity.

RNN Unrolling (Tikz)



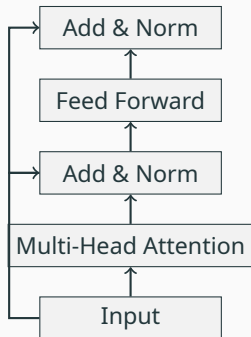
- **Self-Attention Mechanism:**

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- **Why Transformers?**

- Parallelization: No sequential dependency (unlike RNNs).
- Global Receptive Field: Can attend to any part of the sequence.
- Scales exceptionally well to large datasets and models.

Transformer Block (Tikz)



Deep Learning Agents

Perception vs. Action

- **Perception:** Understanding the world.
- **Action:** Deciding what to do.
- DL excels at Perception.
- We can use DL to find features, and then use those features for Action.

- Mapping raw pixels I directly to motor commands u .
- $u = \text{CNN}(I)$.
- No intermediate symbols (e.g., "The ball is at (5,2)").
- Powerful, but hard to debug and verify.

- Why call it “Cloning”? Because we are creating a digital copy of the expert’s behavior.
- **Dataset:** $\mathcal{D} = \{(s_i, a_i)\}_{i=1}^N$
- **Objective:** $\min_{\theta} \sum \mathcal{L}(\pi_{\theta}(s_i), a_i)$
- This is exactly what we do in standard Supervised Learning.

Behavioral Cloning vs. RL

Feature	Behavioral Cloning	Reinforcement Learning
Data Source	Labeled Expert Data	Trial and Error (Rewards)
Algorithm	Backpropagation	Policy Gradient / Q-Learning
Human Effort	High (labeling/expert)	Low (reward function)
Compute	Moderate	Very High
Safety	High (follows expert)	Low (explores randomly)

The “Shift” Problem

1. Expert always stays on the center of the road.
2. Agent learns this.
3. In testing, agent makes a tiny 1-degree error.
4. Agent is now slightly off-center—a state the expert NEVER visited.
5. Agent doesn't know how to recover.
6. Error compounds → Crash.

Solution: Data Augmentation for Agents

- Include “Recovery” data in the dataset.
- Record the expert recovering from a bad state.
- If training a drone, record it being tilted and recovering.
- This teaches the agent how to get back to the “Expert Manifold”.

Hybrid AI: The Best of Both Worlds

- **Neuro-Symbolic AI.**
- **Stage 1:** CNN/MLP identifies entities (e.g., "Food at 5,2", "Wall at 3,3").
- **Stage 2:** A Logic Agent or Rule Engine decides the move.
- **Benefit:** Explainable, verifiable, and needs less data.

Practical Use (Keras 3)

Defining the Model

```
model = keras.Sequential([
    keras.layers.Conv2D(32, 3, activation='relu', padding='same'),
    keras.layers.Flatten(),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dense(4, activation='softmax')
])
```

Compilation and Training

```
model.compile(  
    optimizer='adam',  
    loss='categorical_crossentropy',  
    metrics=['accuracy']  
)  
  
model.fit(X, y, epochs=20, batch_size=32)
```

Inference Loop

```
state = env.reset()
while not done:
    pred = model.predict(state[None, ...], verbose=0)
    action = np.argmax(pred)
    state, _, done = env.step(action)
```

XLA Compilation (JAX)

- JAX uses XLA (Accelerated Linear Algebra) to compile models.
- The first run might be slow (compilation), but subsequent runs are lightning fast.
- Keras 3 handles this automatically when the JAX backend is active.

Summary: Designing your Agent

1. **Observe:** What sensors do you have? (Vector, Image, Text).
2. **Choose Expert:** How would a human or algorithm solve it?
3. **Record:** Capture the state-action pairs.
4. **Train:** Build the appropriate architecture (MLP, CNN, etc.).
5. **Evaluate:** Test in the real environment.
6. **Iterate:** Add data for failed cases.

Common Pitfalls in Training

- **Vanishing Gradient:** Sigmoid/Tanh in deep networks.
- **Exploding Gradient:** Weights become NaN. Solution: Gradient Clipping.
- **Overfitting:** Model memorizes noise. Solution: Dropout, Regularization, Early Stopping.
- **Underfitting:** Model is too simple. Solution: More layers, more neurons.

Hyperparameter Tuning

- **Learning Rate:** The most important hyperparameter.
- **Batch Size:** Affects noise and speed.
- **Epochs:** How long to train.
- **Architectural Choices:** Number of layers, neurons, kernel sizes.
- **Search Strategies:** Grid Search, Random Search, Bayesian Optimization.

Debugging Neural Networks

1. **Overfit a single batch:** If the model can't reach 0 loss on one batch, there's a bug in the code.
2. **Check Init:** Are gradients flowing?
3. **Visualize Predictions:** Don't just trust the accuracy metric.
4. **Monitor Loss Curve:** Look for plateaus or divergence.